⚽ Football Data Platform — End-to-End Data Engineering

Documento de continuidade do projeto — versão baseada no novo domínio de futebol.

Status: arquitetura e roadmap definidos; implementação ainda não iniciada.

1. Objetivo

Construir uma plataforma de dados end-to-end, open source e containerizada, usando dados públicos de futebol. O projeto deve demonstrar competências atuais de Data Engineering: ingestion, ELT, orchestration, modelagem dimensional, data quality, testes, IaC, CI/CD, security, lineage, observability e analytics.

A primeira versão será 100% local com Docker. Depois que estiver estável, o projeto poderá ganhar uma segunda implementação em AWS para demonstrar cloud Data Engineering.

2. Domínio e dataset

Dataset principal: Football-Data.co.uk, inicialmente com dados históricos da Premier League. Os arquivos são disponibilizados em CSV e são simples de baixar e processar. A fonte oferece histórico de resultados e, dependendo da temporada, estatísticas e odds.

Fonte alternativa/expansão: OpenFootball, para ampliar posteriormente para outras ligas e competições. A estratégia inicial é não complicar o projeto com várias fontes: começar com uma liga e depois demonstrar integração de fontes adicionais.

Escopo inicial recomendado:

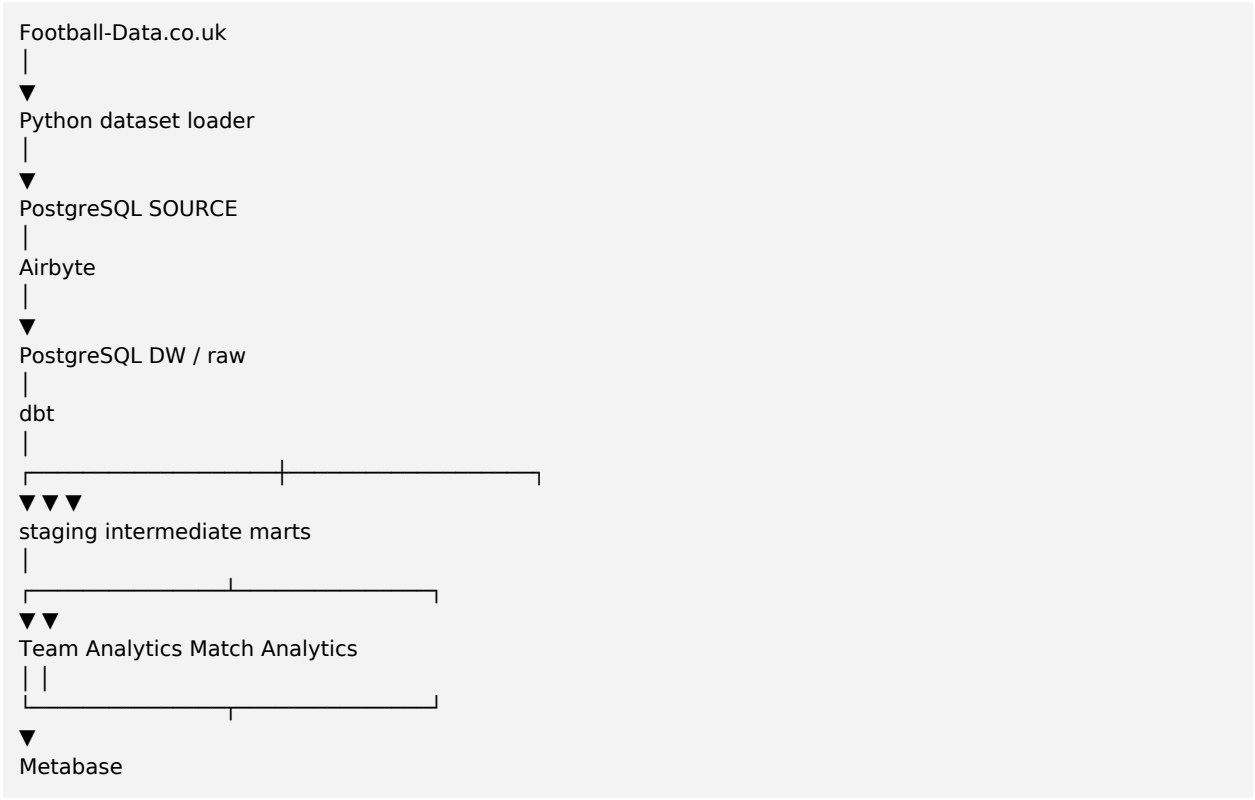
```
Competition: Premier League
History: 1993 → atual
Format: CSV
Primary grain: one row per match
```

3. Stack definida

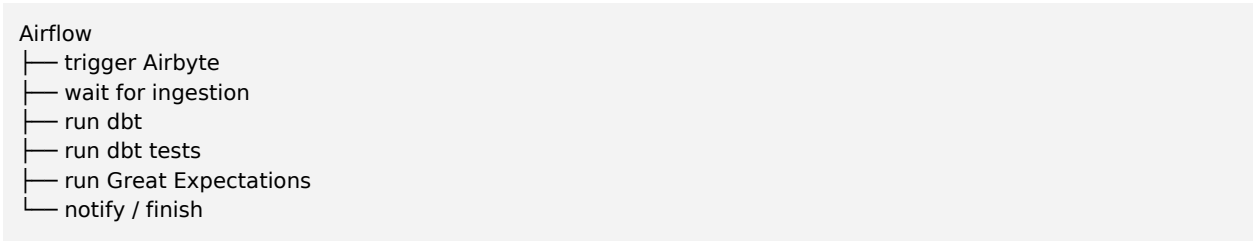
Área	Tecnologia	Função
Versionamento	Git + GitHub	Código, PR, review e histórico
Containers	Docker + Docker Compose	Ambiente local reproduzível
Ingestion	Airbyte	Replicação/ingestion source → raw
Source	PostgreSQL	Banco que recebe o dataset original
Warehouse	PostgreSQL	Raw + staging + intermediate + marts
Transformation	dbt	ELT, modelagem e testes
Orchestration	Apache Airflow	Coordenação do pipeline
Python tests	Pytest	Testes unitários/integration
SQL quality	SQLFluff	Lint de SQL
Data quality	dbt tests + Great Expectations	Regras de qualidade e negócio
IaC	Terraform	Infra/configuração reproduzível
CI/CD	GitHub Actions	Automação de checks e deploy

Área	Tecnologia	Função
Security	Trivy + secret scanning	Vulnerabilidades e secrets
Lineage	OpenLineage + Marquez	Lineage de jobs/datasets
BI	Metabase	Dashboards e consumo analítico
Tooling	pre-commit + Ruff + Makefile	Padronização local

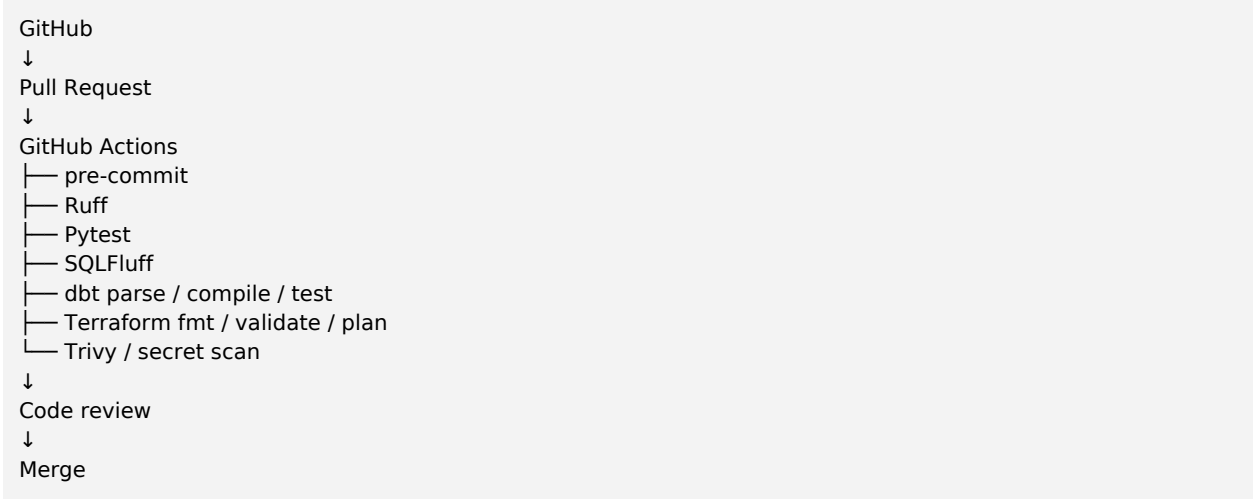
4. Arquitetura



Orchestration:



CI/CD:



```
↓
CD / deployment
```

5. Modelo dimensional

```
dim_team
│
│
dim_date ———————— fct_match ———————— dim_competition
│
│
dim_referee

Optional future dimensions:
dim_stadium
dim_manager
dim_player
dim_country
```

Fato principal — fct_match:

```
match_id
date_key
season_key
competition_key
home_team_key
away_team_key
referee_key
home_goals
away_goals
home_half_time_goals
away_half_time_goals
result
```

Marts derivados:

```
fct_team_season
team
season
matches
wins
draws
losses
goals_for
goals_against
goal_difference
points

fct_daily_metrics
date
matches
goals
avg_goals_per_match
home_win_pct
draw_pct
away_win_pct
```

6. Regras de negócio para demonstrar Data Quality

• home_goals >= 0 e away_goals >= 0

• home_goals e away_goals devem ser inteiros

• result deve ser compatível com o placar: HOME → mandante venceu; DRAW → empate; AWAY → visitante venceu

• home_team e away_team não podem ser NULL

• home_team != away_team, salvo caso documentado de fonte

• date deve ser válida

• season deve ser consistente com a data

• matches_played = wins + draws + losses

• points = 3*wins + 1*draws

• goals_for e goals_against devem bater com os jogos agregados

• chaves de dimensões devem existir

• detectar duplicidade de match_id

• volume esperado por temporada deve estar dentro de faixa razoável

7. Estratégia de ingestion

O dataset será primeiro carregado em um PostgreSQL SOURCE. Isso cria um sistema de origem realista para o Airbyte consumir. O Airbyte será responsável pela replicação para o PostgreSQL DW, na camada raw.

```
CSV / dataset
↓
Python loader
↓
PostgreSQL SOURCE
↓
Airbyte
↓
PostgreSQL DW.raw
```

A transformação deve permanecer no dbt. Regra de arquitetura: Airbyte move dados; dbt transforma dados.

8. Airflow e pipeline operacional

```
DAG: football_data_pipeline

start
↓
trigger_airbyte
↓
wait_for_airbyte
↓
dbt_build
↓
dbt_test
↓
great_expectations
↓
publish_success / notify
```

• retries

• timeouts

• logging

• failure handling

• idempotency

• scheduling

• backfill

9. CI/CD e testes

Check	Ferramenta	Objetivo
Pre-commit	pre-commit	Executar checks básicos antes do commit
Lint Python	Ruff	Qualidade e problemas de Python
Unit tests	Pytest	Testar loader e lógica Python
SQL lint	SQLFluff	Qualidade/formatação SQL
dbt parse	dbt	Validar projeto e dependências
dbt compile	dbt	Validar SQL compilado
dbt test	dbt	Integridade dos modelos
Data quality	Great Expectations	Regras de negócio e qualidade
IaC	Terraform	fmt, validate, plan
Security	Trivy + secret scan	Vulnerabilidades e credenciais expostas

10. Terraform

Terraform entra como Infrastructure as Code. O objetivo é declarar a infraestrutura/configuração que faz sentido, sem substituir o Docker Compose indiscriminadamente.

```
terraform/
├── modules/
│   ├── postgres/
│   ├── airbyte/
│   └── monitoring/
├── environments/
│   ├── dev/
│   └── prod/
└── main.tf

terraform fmt -check
terraform init
terraform validate
terraform plan
tflint / security scan (opcional)
```

Para Airbyte, considerar Terraform para declarar sources, destinations e connections. Para o ambiente local, Docker Compose continua sendo a ferramenta principal de runtime.

11. Lineage e observabilidade

```
Airflow
↓
OpenLineage
↓
Marquez

Exemplo de lineage:
source.matches
↓
raw.matches
↓
stg_matches
```

```
↓
fct_match
↓
dashboard
```

Métricas relevantes:

• pipeline success/failure

• pipeline duration

• rows ingested

• rows rejected

• freshness

• duplicate rate

• NULL rate

• data quality status

12. Metabase — produto final

Dashboard sugerido: Football Analytics.

• Total de partidas

• Total de gols

• Média de gols por partida

• Percentual de vitórias em casa

• Percentual de empates

• Percentual de vitórias fora

• Ranking por temporada

• Gols por temporada

• Performance de times ao longo do tempo

• Home advantage

• Head-to-head

• Sequências de vitórias/sem derrotas

13. Roadmap de implementação

STEP 0 — Definir escopo

Fixar Premier League como primeira competição, período inicial e métricas do dashboard.

STEP 1 — Criar GitHub repo

README, .gitignore, licença, convenções, estrutura inicial.

STEP 2 — Docker base

Subir PostgreSQL SOURCE e PostgreSQL DW com Docker Compose.

STEP 3 — Dataset loader

Criar loader Python para baixar/ler os CSVs e carregar o SOURCE DB.

STEP 4 — Airbyte

Subir/configurar Airbyte e criar conexão PostgreSQL SOURCE → PostgreSQL DW.raw.

STEP 5 — dbt

Configurar projeto e conexão com o DW.

STEP 6 — Staging

Criar modelos staging com tipos, nomes e normalização.

STEP 7 — Intermediate

Criar lógica intermediária e enriquecimentos.

STEP 8 — Marts

Criar fct_match, fct_team_season e métricas analíticas.

STEP 9 — dbt tests

Adicionar unique, not_null, relationships, accepted_values e testes customizados.

STEP 10 — Metabase

Criar dashboard e validar que o produto final responde perguntas reais.

STEP 11 — Airflow

Orquestrar Airbyte → dbt → tests → DQ.

STEP 12 — Great Expectations

Adicionar validações de volume, freshness, ranges e regras de negócio.

STEP 13 — Terraform

Declarar infraestrutura/configuração relevante e integrar validações ao CI.

STEP 14 — GitHub Actions

Implementar CI completo: lint, unit, SQL, dbt, Terraform e security.

STEP 15 — Security

Trivy, secret scanning e revisão de permissões/secrets.

STEP 16 — CD

Automatizar deployment do ambiente conforme estratégia definida.

STEP 17 — Lineage

Adicionar OpenLineage + Marquez.

STEP 18 — Observability

Adicionar métricas, logs, freshness e alertas básicos.

STEP 19 — Expandir domínio

Adicionar outras ligas/competições ou uma segunda fonte como OpenFootball.

STEP 20 — Versão AWS

Mapear componentes para AWS e implementar segunda versão cloud.

14. Estrutura sugerida do repositório

```
football-data-platform/
├── .github/
│ └── workflows/
│ ├── ci.yml
│ ├── cd.yml
│ └── terraform.yml
├── airflow/
│ ├── dags/
│ └── requirements.txt
├── airbyte/
│ └── connections/
├── dbt/
│ ├── models/
│ │ ├── staging/
│ │ ├── intermediate/
│ │ └── marts/
│ ├── tests/
│ ├── macros/
│ ├── seeds/
│ └── dbt_project.yml
├── ingestion/
│ ├── src/
│ └── tests/
├── data_quality/
│ └── expectations/
├── terraform/
│ ├── modules/
│ └── environments/
├── docker/
├── docker-compose.yml
├── Makefile
├── pyproject.toml
├── .pre-commit-config.yaml
├── .sqlfluff
├── README.md
└── architecture.png
```

15. Princípios técnicos que precisam aparecer no projeto

• Idempotência: rerun não cria duplicação ou estado inconsistente.

• Incremental loading: quando fizer sentido, evitar processar toda a história a cada execução.

• Data contracts: schema esperado deve ser explícito.

• Schema evolution: alterações de schema precisam ser detectadas/tratadas.

• Backfill: permitir reprocessar uma temporada/período.

• Separation of concerns: Airbyte move, Airflow orquestra, dbt transforma, DQ valida.

• Reprodutibilidade: Docker + IaC + versionamento.

• Security: nunca versionar secrets.

• Observability: saber se o pipeline rodou e se os dados estão saudáveis.

16. O que evitar no início

• Não começar com Kubernetes/EKS.

• Não adicionar várias ligas/fontes antes de o pipeline Premier League funcionar.

• Não usar Terraform apenas por obrigação; aplicar onde houver benefício real.

• Não duplicar a mesma validação em várias ferramentas sem motivo.

• Não colocar transformação pesada no Airbyte.

• Não começar pela AWS; primeiro estabilizar a versão local.

17. Critérios de conclusão

• Ambiente local sobe via Docker.

• Dataset Premier League é carregado no PostgreSQL SOURCE.

• Airbyte replica para raw.

• Airflow executa o pipeline completo.

• dbt constrói staging/intermediate/marts.

• Data quality detecta violações de regras de negócio.

• Metabase mostra dashboard funcional.

• GitHub Actions bloqueia PR com falhas.

• Terraform valida/reproduz a infraestrutura definida.

• Lineage pode ser visualizado.

• README permite que outra pessoa execute o projeto.

• Existe documentação de decisões, trade-offs e troubleshooting.

18. Contexto para continuar em novo chat

Prompt de continuidade: “Quero continuar meu projeto Football Data Platform. É um projeto end-to-end de Data Engineering, inicialmente 100% local/open source, usando Football-Data.co.uk/Premier League como dataset principal. A stack é Docker, GitHub, Airbyte, PostgreSQL source + PostgreSQL DW, dbt, Airflow, Great Expectations, Terraform, GitHub Actions, Ruff, Pytest, SQLFluff, Trivy, OpenLineage/Marquez e Metabase. O documento de contexto define a arquitetura, modelo dimensional, testes e roadmap. Quero continuar pelo STEP X.”

Estratégia de evolução: primeiro MVP funcional; depois production-like; por fim expansão para outras ligas/fontes e uma implementação AWS.

19. Visão final do projeto

```
FOOTBALL DATA PLATFORM

Football-Data.co.uk / OpenFootball
↓
PostgreSQL SOURCE
↓
Airbyte
↓
PostgreSQL RAW
↓
dbt
↓
staging → intermediate → marts
↓
┌──────────┴──────────┐
▼ ▼
Data Quality Metabase
▲
│
Airflow
│
└── orchestration

GitHub → GitHub Actions → CI/CD
Terraform → IaC
OpenLineage → Lineage
Trivy → Security
Docker → Reproducibility
```

Este documento substitui o plano anterior baseado em NYC Taxi. O domínio oficial do projeto passa a ser futebol.